



User Guide

Delphi App Translation Studio



Version 1.0

Last changed: August 30, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

1. About This Guide	1
2. System and Developer Prerequisites	2
3. Download the Studio from GitHub	3
4. Build and Start the Studio	4
5. Prepare the Target Delphi Application	4
6. The dependencies Folder and Delphi Search Path	5
7. Files Stored Under %LOCALAPPDATA%	7
8. Starting the Studio and Keyboard Operation	10
9. Setup Wizard Navigation and Safety	11
10. Wizard Step 1 - Welcome	12
11. Wizard Step 2 - Delphi Project	13
12. Wizard Step 3 - Deployment	14
13. Wizard Step 4 - Languages	16
14. Wizard Step 5 - Translation Service	17
15. Wizard Step 6 - Scan Project	19
16. Wizard Step 7 - Review and Authorize	20
17. Localization Review Window	21
18. Wizard Step 8 - Process and Finish	22
19. Maintenance Studio Overview	23
20. Maintenance Page 1 - Project	24
21. Maintenance Page 2 - Scan	25
22. Maintenance Page 3 - Translate	26
23. Maintenance Page 4 - Validation	27
24. Maintenance Page 5 - Export	28
25. Maintenance Page 6 - Integration	29
26. Maintenance Page 7 - Provider Settings	31

27. DeepL and Google Setup	32
28. Runtime Integration in VCL and FMX	33
29. Build, Deploy, and Verify	33
30. RTL, Layout, HTML, and Dynamic Text	34
31. Security, Privacy, and Recovery	34
32. Troubleshooting	35
33. Complete First-Time Checklist	36
34. Quick Path Reference	38

1. About This Guide

This is the complete operating guide for Delphi App Translation Studio. It is written from the current Pascal source, FMX form definitions, package projects, scanner contracts, provider code, workspace code, and the verified application behavior as of August 30, 2026. It covers both VCL and FireMonkey target applications.

The guide explains installation, project preparation, every Studio and Setup Wizard screen, every visible operating control, provider security, catalogs, offline packs, component integration, compiler paths, deployment, verification, right-to-left behavior, recovery, and troubleshooting. A developer should be able to start with a GitHub download and finish with a translated Win32 or Win64 application without relying on undocumented steps.

The product is currently distributed as source. There is no supported binary installer. Use a disposable copy of a target application until the translated result has passed the verification checklist in Chapter 33.

Core safety rule. Scanning reads the selected Delphi project. Final Setup Wizard processing creates a safety ZIP, workspace files, runtime packs, review artifacts, and a component kit, but the current recommended workflow does not rewrite the target Pascal, DFM, FMX, DPR, or DPROJ files.

1.1 What the Studio produces

Product	Purpose	Primary location
Development catalog	Editable source/translation records, locale facts, review state, context, checksums, and scan provenance.	%LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\<ApplicationId>\Development
Runtime language pack	Compact, validated JSON read by the target application without Internet access.	%LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\<ApplicationId>\Languages
Project glossary	Approved project-specific terminology applied before general machine-translation wording.	%LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\<ApplicationId>\Glossaries
Localization Review package	HTML review, glossary candidates, proposals, audit findings, and supporting JSON.	export\localization-review\<ApplicationId>\<locale>

Product	Purpose	Primary location
Component integration kit	Framework-appropriate runtime/component units, source pack, translated packs, manifest, deployment script, README, and completion report.	export\component-integration\<ApplicationId>
Safety backup	Timestamped ZIP made before final processing.	The backup path is displayed in Wizard progress and the completion report.

1.2 Supported and excluded scope

- Supported target frameworks: Delphi VCL and FireMonkey (FMX).
- Supported target platforms: Windows Win32 and Win64; Debug and Release configurations may be built locally.
- Supported online providers during development: DeepL API and Google Cloud Translation Basic v2.
- The deployed application is offline. It does not contain a provider API key and does not call DeepL or Google.
- macOS, iOS, Android, Linux, C++Builder, and runtime cloud translation are outside the present release scope.
- Dynamic strings assembled from live data may require an application-authored semantic key or explicit runtime application; a static scanner cannot infer every possible runtime sentence.

2. System and Developer Prerequisites

Requirement	Required state
Windows	A current 64-bit Windows development system with permission to write the selected project copy, Local AppData, and the Studio export folder.
RAD Studio	RAD Studio 13 Florence. The verified toolchain is under C:\Program Files (x86)\Embarcadero\Studio\37.0\bin.

Requirement	Required state
Delphi project	A saved VCL or FMX .dproj/.dpr that builds successfully before localization.
Source form format	Text DFM/FMX resources are preferred for auditability. Save all designer changes before scanning.
Internet	Required only while testing a provider connection or translating unresolved entries.
Provider account	A DeepL API Free/Pro key or Google Cloud Translation API key. DeepL is the recommended default.
Version control	Git or an equivalent recoverable baseline for both the Studio source and the disposable target copy.
Do not start with the production copy. Make a pristine backup and a separate test copy of the Delphi application. Confirm the test copy compiles before adding DAT components or language packs.	

3. Download the Studio from GitHub

The official repository is <https://github.com/tmartindub/DelphiAppTranslationStudio>. The entire repository is the required source distribution because the project, packages, runtime units, images, localization files, tests, tools, and documentation use its folder structure.

Do not download one .pas file, one .dproj, or a component BPL by itself. A partial download omits package dependencies, runtime units, localization resources, and build output paths.

3.1 Download a ZIP

1. Open the repository URL in a browser.
2. Use the branch selector to choose the published beta/release branch specified by the project owner. For the current stabilization candidate, that branch is codex/total-stabilization-release; after it is merged or released, use the published release tag or main branch named by the repository.
3. Choose Code, then Download ZIP.
4. Save the ZIP to a normal download folder. Do not run or build from the browser preview or from inside the ZIP.
5. Extract the complete archive to a short, writable development path such as C:\New Delphi Projects\Delphi App Translation.

6. Open the extracted root and confirm that DelphiAppTranslationStudio.dproj, source, packages, Localization, docs, tools, and images and icons are present.

3.2 Clone with Git

```
git clone https://github.com/tmartindub/DelphiAppTranslationStudio.git
```

Cloning is preferred for developers who will receive updates. Keep local work on a separate branch, review upstream changes, and never commit provider credentials or proprietary target catalogs.

3.3 What not to download

- Do not treat bin as an installer. Local executables and BPLs must match the active RAD Studio toolchain.
- Do not install a .dpk through Install Component. Build the package and add the Win32 design BPL through Component > Install Packages.
- Do not rely on an older source-distribution ZIP if its date predates the selected repository branch. The full branch archive is authoritative until a signed release package is published.

4. Build and Start the Studio

1. Start RAD Studio 13 Florence.
2. Open DelphiAppTranslationStudio.dproj from the extracted repository root.
3. Choose Win32 or Win64 and Debug or Release. Win32 Release is the simplest first build and is also the platform used for Delphi design-time packages.
4. Choose Project > Build DelphiAppTranslationStudio.
5. Run the executable from bin\<Platform>\<Configuration>, for example bin\Win32\Release\DelphiAppTranslationStudio.exe.

The verified command-line environment is initialized by C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rvars.bat. The Win32 compiler is C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\dcc32.exe. These toolchain paths identify Delphi itself; they are different from the target project's DAT unit Search Path described in Chapter 6.

Build output. Executables go to bin\<Platform>\<Configuration>; DCUs go under dcu. Keep the repository folder structure intact so relative package, localization, export, and documentation paths remain valid.

5. Prepare the Target Delphi Application

1. Create a pristine backup and a separate test copy of the target application.

2. Open the test copy in RAD Studio and build every platform/configuration you intend to support. Correct pre-existing build errors before localization.
3. Build the matching DAT runtime and design packages from the Studio repository.
4. In RAD Studio choose Component > Install Packages > Add and select DATLanguageManagerVCLDesign.bpl or DATLanguageManagerFMXDesign.bpl from bin\packages\Win32\Release.
5. Open the target application's primary form in the Form Designer.
6. Place one TDATVCLLanguageManager or TDATFMXLanguageManager on the primary form.
7. Set ApplicationId to the exact Delphi project name, LanguagesFolder to Localization\Languages, and SourceLanguage to the source locale, normally en-US.
8. Place one TDATVCLLanguageComboBox or TDATFMXLanguageComboBox and set its LanguageManager property to the manager. A designer-authored connected Language menu is an alternative, but the user needs a visible selection mechanism.
9. Add any supporting Language label or menu captions in the designer, then choose File > Save All.
10. Close the target project before the Setup Wizard's final-processing step.

One manager. The primary form owns the single language manager. Ordinary forms do not need another manager. The manager retranslates open forms and applies the active pack to later forms through the framework-appropriate lifecycle.

6. The dependencies Folder and Delphi Search Path

The Studio's generated component kit contains a ComponentSource folder. The Studio does not silently create a dependencies folder inside every target project. A project-local dependencies\DelphiAppTranslation\source folder is an optional, recommended vendoring arrangement when the developer wants the target project to compile independently of the Studio export folder.

Create the dependencies folder only in the disposable/controlled target project. Populate it by copying the complete current ComponentSource set generated for that target framework. Do not copy a hand-picked subset: a set that compiles today may fail after another runtime unit begins using a shared dependency.

6.1 Recommended target layout

```
<Target Project>\dependencies\DelphiAppTranslation\source
<Target Project>\Localization\Languages
```

1. Complete the Wizard or generate a component kit from Maintenance Studio.
2. Open export\component-integration\<ApplicationId>\ComponentSource.
3. Create <Target Project>\dependencies\DelphiAppTranslation\source if it does not exist.

4. Copy every .pas file from ComponentSource into that source folder, replacing the previous DAT set as one versioned unit.
5. Commit the dependency units with the target test project if the project's licensing/distribution policy permits vendoring them.
6. Whenever the Studio runtime is updated, regenerate the kit and refresh the whole set together; do not mix files from different builds.

6.2 Canonical framework unit set

Shared units	VCL target adds	FMX target adds
DAT.Core.Atomi cFile; DAT.Core.Diagn ostics; DAT.Runtime.La nguagePack; DAT.Runtime.Pr eference; DAT.Runtime.M anager; DAT.Runtime.La youtOverrides; DAT.Runtime.Spl ashTranslation; DAT.Runtime.Te mplateRewrite; DAT.Component s.Core	DAT.Runtime.VCL; DAT.Runtime.SplashTranslation.VCL; DAT.Runtime.TemplateRewrite.VCL; DAT.Components.VCL; DAT.Components.VCL.LanguageSelector	DAT.Runtime.FMX; DAT.Runtime.SplashTranslation.FMX; DAT.Runtime.TemplateRewrite.FMX; DAT.Components.FMX; DAT.Components.FMX.LanguageSelector

6.3 Set the exact compiler Search Path

1. Open the target project in RAD Studio.
2. Choose Project > Options.
3. At the top of the Options dialog choose All configurations and All platforms unless a platform intentionally uses a different dependency copy.
4. Open Building > Delphi Compiler.
5. Locate Search path.

6. Append `.\dependencies\DelphiAppTranslation\source`. Preserve every existing entry and preserve inherited macros such as `$(DCC_UnitSearchPath)`.
7. Choose Save, then OK.
8. Build Win32 Debug, Win32 Release, and any supported Win64 configurations. A clean build verifies that no DAT unit is being found accidentally from an unrelated global path.

Recommended portable entry: `.\dependencies\DelphiAppTranslation\source`

Example absolute entry: `C:\New Delphi Projects\VCL2FMXConverterV5 - Test\dependencies\DelphiAppTranslation\source`

Alternative. A project may point directly to `export\component-integration\<ApplicationId>\ComponentSource`. That works, but the target then depends on a generated Studio export path. The project-local dependency folder is easier to archive, clone, and build on another development system.

7. Files Stored Under %LOCALAPPDATA%

%LOCALAPPDATA% expands to the current Windows user's Local AppData folder, normally `C:\Users\<WindowsUser>\AppData\Local`. These files are per-user working state. They are deliberately outside Program Files and outside the target project so normal Windows permissions do not require the executable folder to be writable.

Path or file	Contents	Operational meaning
%LOCALAPPDATA%\DelphiAppTranslationStudio\provider-settings.json	Provider name, DeepL plan, remember-credential choice, request timeout, and batch size.	Contains no API key. Defaults are DeepL, API Free, remember enabled, 30 seconds, and 40 strings per request.
%LOCALAPPDATA%\DelphiAppTranslationStudio\language.ini	Studio interface language preference.	Lets the Studio reopen in the selected interface language.

Path or file	Contents	Operational meaning
%LOCALAPPDAT A% \DelphiAppTranslationStudio\Workspaces\<ApplicationId>\Development\<ApplicationId>.<locale>.translation-project.json	Editable development catalog.	Authoritative scan/translation/review workspace for that application and locale.
... \Languages\<locale>.json	Validated runtime pack.	Copied into component kits and deployed beside target executables.
... \Languages\<locale>.layout-proposal.json	Reviewed layout decisions when present.	Accepted direction/fitting/relative layout rules can be embedded in the runtime pack.
... \Glossaries\<ApplicationId>.<locale>.glossary.json	Approved project-specific terms.	Overrides general machine wording for exact project terminology.
... \Deployment\deployment-destinations.json	Optional application destination folders.	Restores portable/network/USB deployment destinations on the next Wizard run.
%LOCALAPPDAT A%\<Target ApplicationId>\language.ini	The target application's selected language.	Created by the DAT manager unless PreferenceLocation is customized.

7.1 Atomic recovery files

- A successful replacement may retain one .previous file containing the last valid text.
- A temporary sibling ending in .tmp is used during atomic publication and should not remain after a successful operation.
- An invalid current file may be quarantined with a .corrupt-<timestamp>-<identifier> name while the prior valid file is recovered.
- Do not delete recovery files while diagnosing a failed catalog, settings, pack, glossary, or preference load. Copy them first and compare timestamps.

7.2 Where API keys are stored

Remembered keys are Windows Generic Credentials, not JSON files. The target names are DelphiAppTranslationStudio/Providers/DeepL and DelphiAppTranslationStudio/Providers/Google Cloud Translation. An unchecked Remember option keeps the key for the current process only and removes any stored credential for that provider.

Use Windows Credential Manager to confirm or remove remembered credentials. Never paste a key into source, catalogs, bug reports, screenshots, documentation, logs, deployment scripts, or Git.

8. Starting the Studio and Keyboard Operation

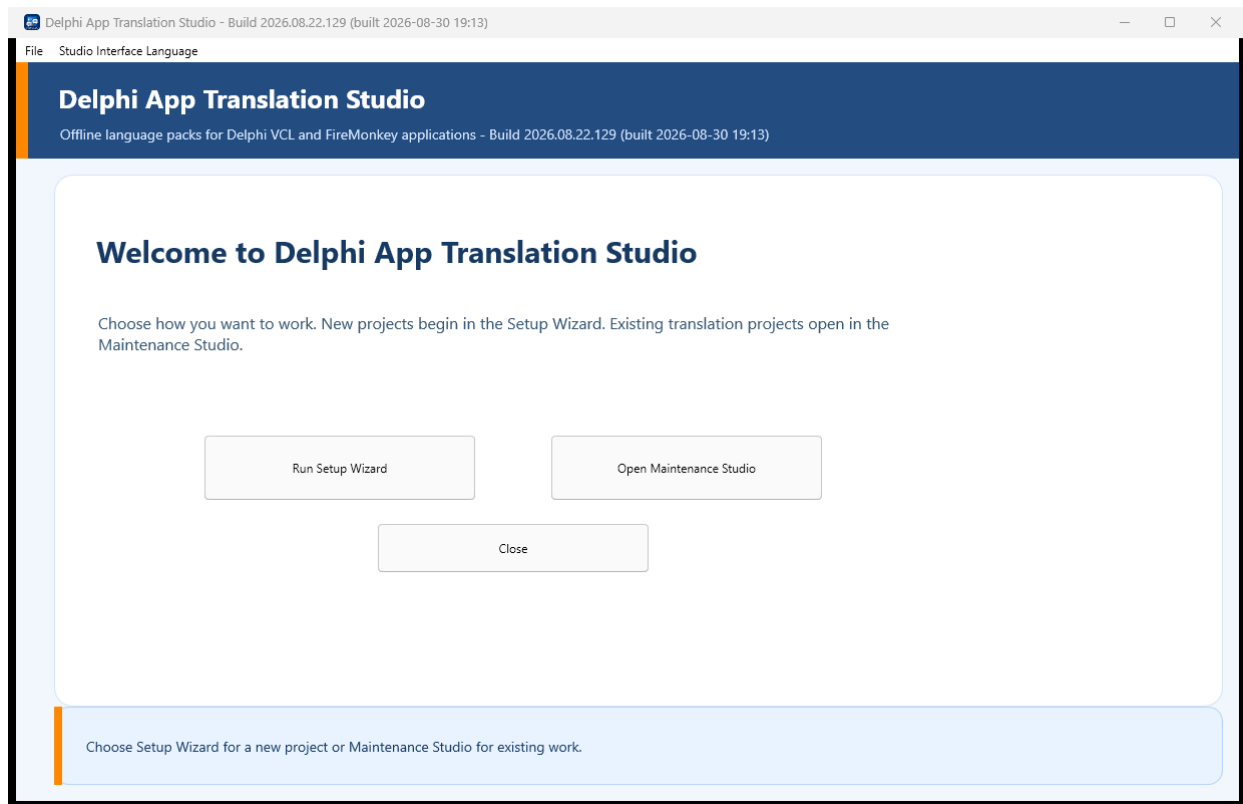


Figure 8-1. Current landing screen.

The landing screen separates first-time setup from maintenance. Run Setup Wizard is the default action, so Enter starts the guided path. Tab and Shift+Tab move between controls; Space activates a focused button or check box; Alt+Down or F4 opens a focused combo box; arrow keys move through its choices; Enter accepts; Escape closes a list or cancels the current dialog where safe.

Control or area	What it does	When to use it
Run Setup Wizard	Opens the eight-step guided workflow for a new or updated localization setup.	Use for the first language or when you want one controlled scan-to-kit pass.
Open Maintenance Studio	Opens the seven-page direct workflow.	Use for existing catalogs, manual review, validation, export, integration, or provider settings.
Close	Closes the Studio.	Use when no operation is running.

Control or area	What it does	When to use it
File > Exit	Normal application exit path.	Use from Maintenance Studio; an active operation observes safe cancellation boundaries.
Studio Interface Language	Changes the Studio's own UI language.	This does not change the source or target language of an opened Delphi project.



Figure 8-2. Landing-screen action buttons.

9. Setup Wizard Navigation and Safety

The Setup Wizard has eight steps: Welcome, Delphi Project, Deployment, Languages, Translation Service, Scan Project, Review Authorize, and Process Finish. Completed steps may be revisited from the left rail until final processing begins.

Next is the default button on ordinary pages; Enter therefore advances only when the current page is valid. Back returns to the previous page. Cancel closes before processing. During processing it becomes Stop and requests cancellation at a safe boundary. After successful completion Back and Cancel are hidden, leaving Finish as the clear completion action. If processing stops, Cancel remains available so the developer can close after reading the STOPPED result.

Control or area	What it does	When to use it
Step rail	Shows progress and allows revisiting completed steps before authorization.	Select a completed step to correct a choice.
Back	Returns one step without discarding already entered values.	Available until final processing begins; hidden on the completed Finish page.
Next	Validates the current step and advances.	The default Enter action on Steps 1-6.

Control or area	What it does	When to use it
Begin Final Processing	Starts the authorized single pass.	Appears on Review Authorize after required confirmations.
Cancel / Stop	Closes before work or requests cancellation during work.	Cancellation is honored at a safe boundary; it is not a forced thread termination.
Finish	Closes a successfully completed Wizard.	The only completion button after success.

10. Wizard Step 1 - Welcome

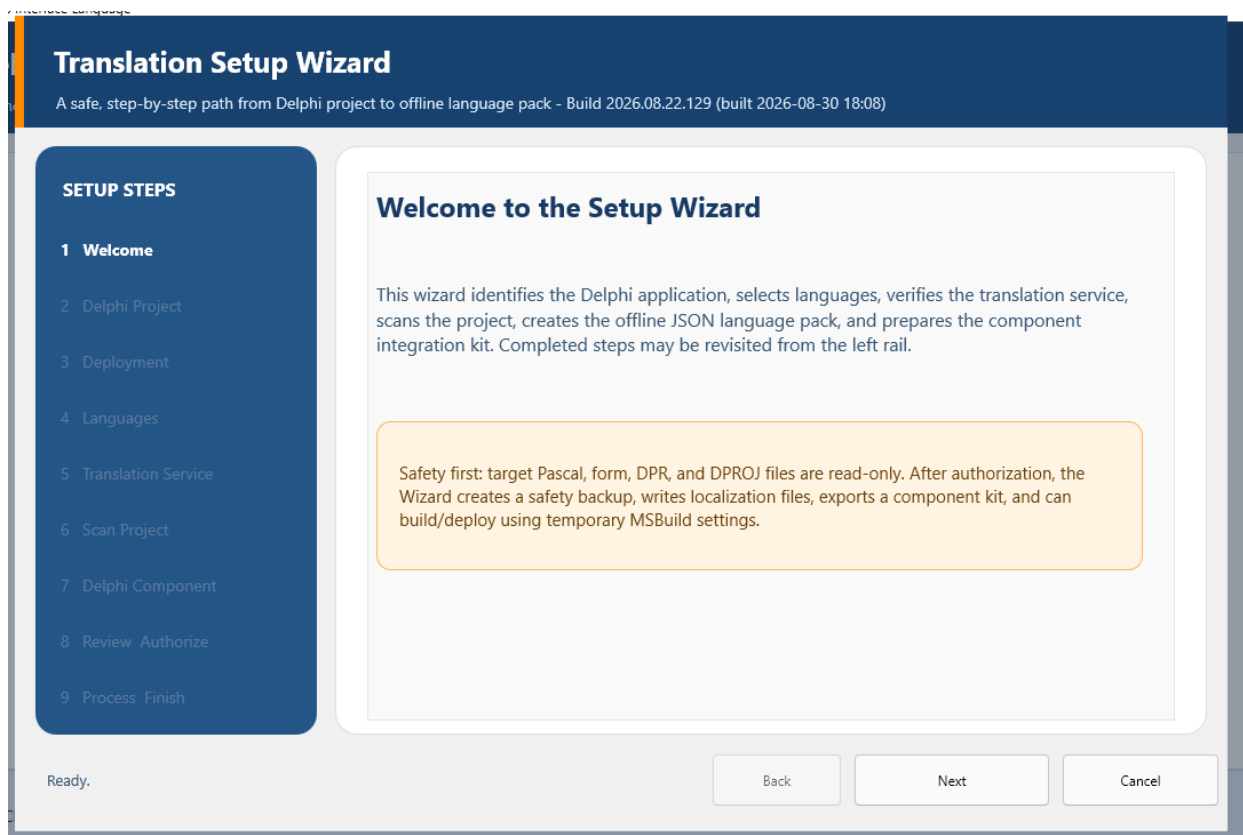


Figure 10-1. Welcome page.

Welcome summarizes the controlled workflow and the safety boundary. It makes no project change. Press Enter or choose Next to continue. Back is disabled because there is no earlier Wizard page; Cancel returns to the Studio.

11. Wizard Step 2 - Delphi Project

Translation Setup Wizard
A safe, step-by-step path from Delphi project to offline language pack - Build 2026.08.22.129 (built 2026-08-30 19:13)

SETUP STEPS

- Welcome
- Delphi Project**
- Deployment
- Languages
- Translation Service
- Scan Project
- Review Authorize
- Process Finish

Select the Delphi project

Choose the .dproj or .dpr file. The Wizard reads project metadata now; no target file changes until you explicitly authorize final processing.

No Delphi project selected Browse...

Detected Application ID
Select a Delphi project Copy ID

Project details will appear here.

Translation workflow
Automatic (Recommended) ▼ Select a project and target language to detect existing work.

Back Next Cancel

Figure 11-1. Delphi Project page.

Control or area	What it does	When to use it
Project path	Displays the selected .dproj or .dpr.	Treat it as read-only confirmation after Browse.
Browse	Selects the Delphi project.	Choose the saved disposable/test copy, not the production original.
Detected Application ID	Shows the exact project identity used by catalogs and packs.	It must match the manager component's ApplicationId.
Copy ID	Copies the detected ID.	Use when setting the manager in Delphi's Object Inspector.
Project details	Reports framework, targets, form resources, and source units.	Verify VCL/FMX and expected project scale before continuing.

Control or area	What it does	When to use it
Translation workflow	Automatic recommends create/update based on existing workspace state; explicit choices can create or update.	Use Update Existing Translation to preserve reviewed/approved entries when a compatible catalog exists.

Application identity is a contract. Catalog applicationId, runtime-pack applicationId, manager ApplicationId, workspace folder, and generated kit identity must agree. Changing the identity creates a different localization workspace.

12. Wizard Step 3 - Deployment

Translation Setup Wizard
A safe, step-by-step path from Delphi project to offline language pack - Build 2026.08.22.129 (built 2026-08-30 19:13)

SETUP STEPS

- Welcome
- Delphi Project
- Deployment**
- Languages
- Translation Service
- Scan Project
- Review Authorize
- Process Finish

Choose application destinations

Build-output folders are detected automatically. Add any separate installed, portable, network, or USB application folders here. Final processing and Wizard-initiated builds deploy JSON language packs to every available destination. This step is optional.

☐ Authorize creating or replacing the deployed application EXE in these folders

Add Application Folder... Remove Selected

No separate destinations are configured. Detected Win32 and Win64 build outputs will still deploy automatically. JSON packs only; the deployed executable will not be replaced.

Back Next Cancel

Figure 12-1. Deployment page.

Control or area	What it does	When to use it
Destination list	Stores separate installed, portable, network, or USB application folders.	Leave empty when normal detected build outputs are sufficient.

Control or area	What it does	When to use it
Add Application Folder	Adds one full application destination.	Select the folder containing or intended to contain the deployed executable.
Remove Selected	Removes only the selected optional destination.	Does not delete the folder or application.
Authorize creating or replacing the deployed EXE	Allows the optional build/deploy action to place an executable in listed destinations.	Leave clear when only JSON packs should be deployed.

Final processing always attempts pack deployment to existing detected build outputs. Separate destinations are optional. A removable or network destination that is unavailable is reported; it is not silently treated as a successful deployment.

13. Wizard Step 4 - Languages

Figure 13-1. Languages page.

Control or area	What it does	When to use it
Source language	The language of the saved Delphi UI text.	Normally English (United States) [en-US]. Do not use the desired output language here.
Target language	The locale being created or updated.	Select explicitly; the Studio does not silently assume German or another target.
Native language name	The display name shown to users in that language.	Auto-filled from locale facts and editable when the product requires a preferred name.

Control or area	What it does	When to use it
Direction	Derived from locale facts in the catalog/runtime contract.	Arabic, Hebrew, Persian, Urdu, Pashto, and other RTL locales follow the common RTL path.

Selecting the target locale also establishes date, time, decimal, thousands, currency, and direction facts. Existing compatible catalogs restore their saved locale facts without a programmatic combo-box event overwriting them.

14. Wizard Step 5 - Translation Service

Translation Setup Wizard
A safe, step-by-step path from Delphi project to offline language pack - Build 2026.08.22.129 (built 2026-08-30 19:02)

SETUP STEPS

- Welcome
- Delphi Project
- Deployment
- Languages
- Translation Service**
- Scan Project
- Review Authorize
- Process Finish

Connect the translation service

The Studio uses Google Cloud Translation or DeepL while the developer is online. The completed target application remains fully offline.

Provider: DeepL (selected)

DeepL plan: API Free (selected)

API key:

☒ Remember securely on this computer

Buttons: Save / Replace Key, Test Connection, Back, Next, Cancel

Note: A saved key is available in Windows Credential Manager

Figure 14-1. Translation Service page with DeepL selected.

Control or area	What it does	When to use it
Provider	Selects DeepL or Google Cloud Translation.	DeepL is the recommended and default provider.

Control or area	What it does	When to use it
DeepL plan	Selects API Free or API Pro endpoint.	Must match the DeepL key/account plan; ignored for Google.
API key	Accepts a new key; it is masked.	A saved key need not be retyped.
Remember securely	Writes a Windows Generic Credential for the selected provider.	Clear it for session-only use; clearing also removes that provider's stored key when saved.
Save / Replace Key	Stores the new key or keeps it for the session according to Remember.	Use after pasting or replacing a key.
Test Connection	Performs a bounded provider request using the selected plan/settings.	A result or timeout is reported; the UI remains cancellable.
Status text	States whether a saved key exists and reports test results.	A saved-key message does not reveal the key.
Keys are provider-specific. Switching provider does not reuse the other provider's key. A DeepL credential can exist at the same time as a Google credential.		

15. Wizard Step 6 - Scan Project

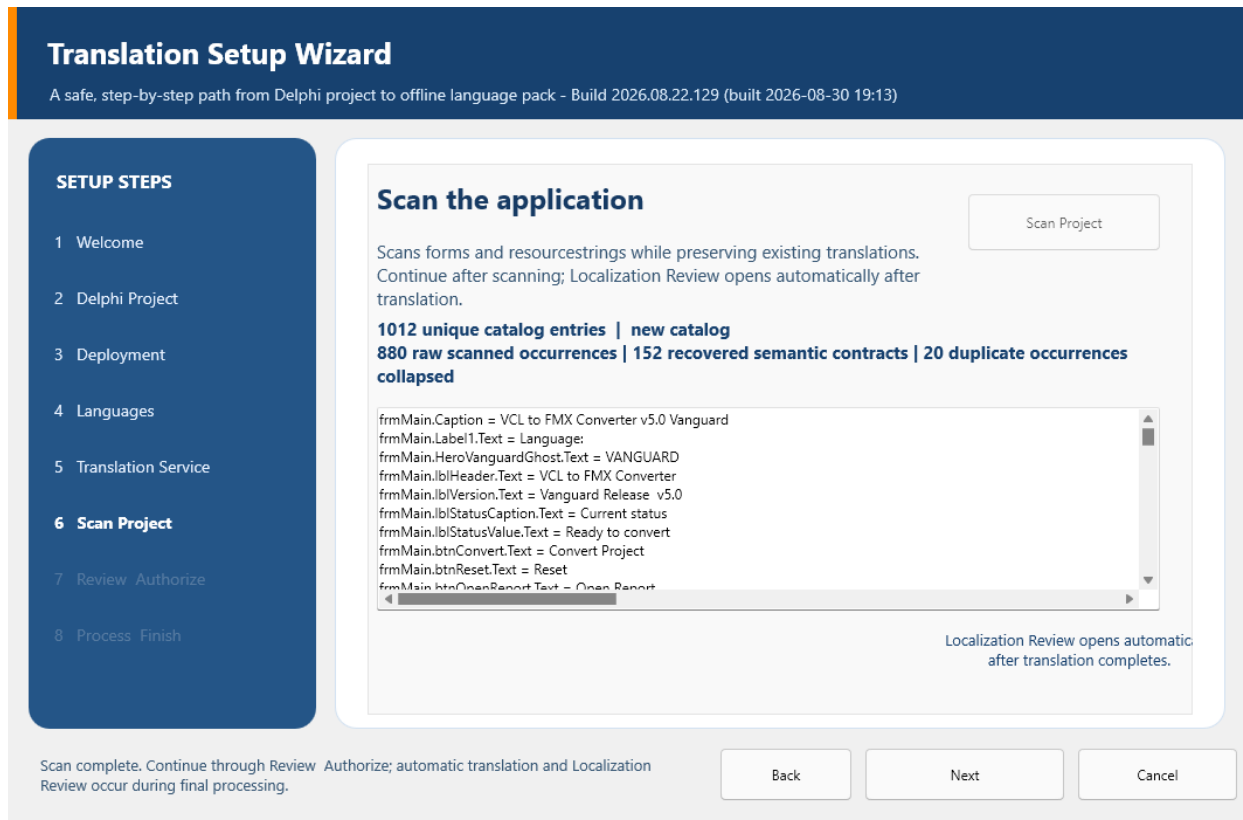


Figure 15-1. Scan page after the verified 1,012-entry scan.

Scan Project reads saved forms and supported source/resource contracts. The current summary distinguishes unique catalog entries from raw observations. Equivalent duplicate occurrences are collapsed; semantic contracts recovered from compatible prior work remain identified; conflicting duplicate keys remain visible for correction.

The list is evidence, not a second catalog editor. Review project/form/source counts and sample entries. If the count is unexpectedly small, confirm that the correct project copy was selected, forms are saved, expected source directories are in the project, and any external JSON prose has an explicit dat-translatable-resources.json manifest.

Control or area	What it does	When to use it
Scan Project	Runs or reruns the inventory.	Rerun after saving target UI/source changes.

Control or area	What it does	When to use it
Summary	Reports unique entries, raw occurrences, recovered semantic contracts, and collapsed duplicates.	Use these categories when comparing Maintenance and Wizard scans.
Results memo	Lists key, source text, and representative observations.	Spot-check the first entries and search later in the development catalog for full details.
Next	Continues to Review Authorize after a successful scan.	Automatic translation and Localization Review occur during final processing, not while merely viewing this page.

16. Wizard Step 7 - Review and Authorize

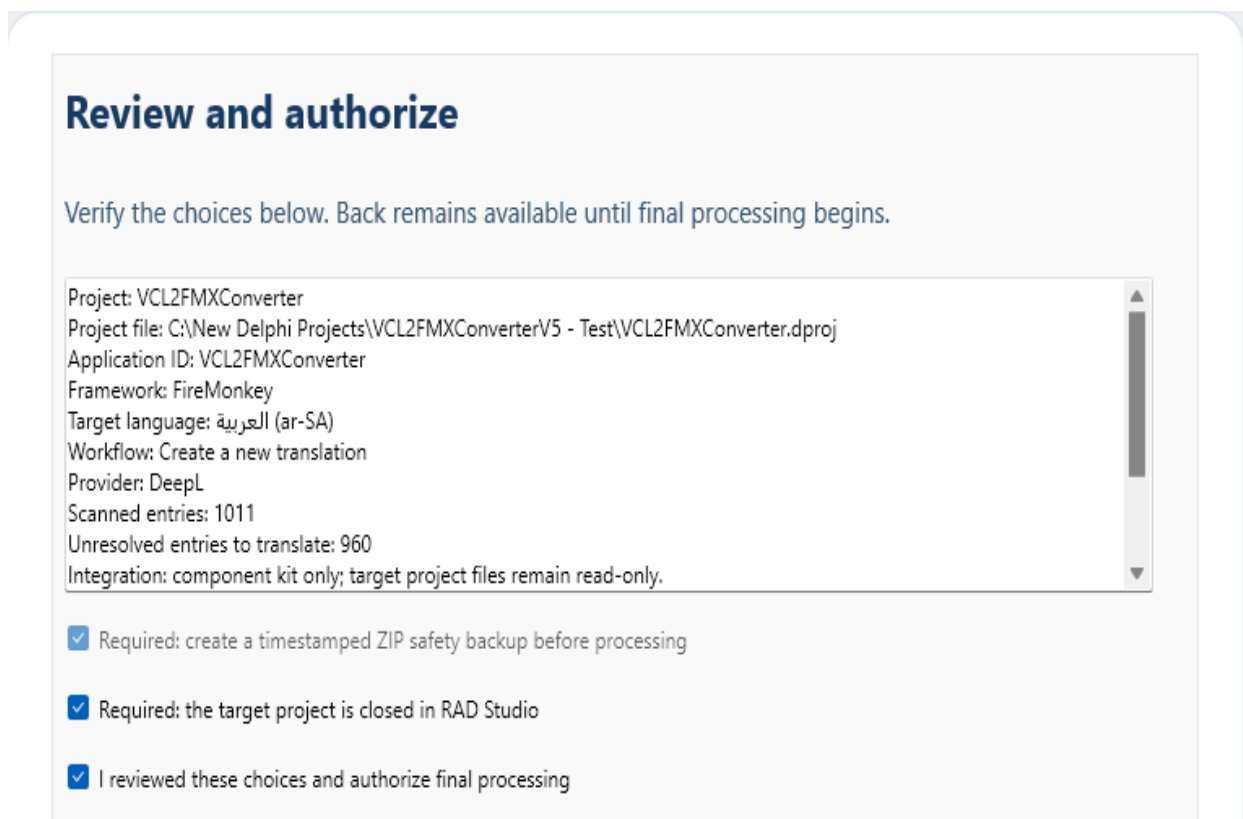


Figure 16-1. Review and Authorize content area; the current Wizard uses this same confirmation contract in Step 7.

Control or area	What it does	When to use it
Review memo	Summarizes project, application ID, framework, target locale, workflow, provider, scan count, unresolved entries, integration mode, and destinations.	Read it from top to bottom before authorizing.
Required ZIP backup	Confirms the mandatory pre-processing safety archive.	Required and selected by design.
Target project is closed	Confirms RAD Studio is not holding the target project during the controlled pass.	Close the target project, then check it.
Authorize final processing	Records the developer's explicit approval.	Check only after all earlier choices are correct.
Begin Final Processing	Starts backup, translation, review, validation, export, kit generation, builds/deployment where configured, and the completion report.	After this click, Back and the rail are disabled; Stop remains available at safe boundaries.

17. Localization Review Window

Localization Review opens automatically after provider translation and before the final runtime pack is exported. It is part of the same Wizard processing pass. Closing it resumes validation, export, component-kit generation, and deployment.

Tab	Purpose	Primary actions
Audit	Summarizes readiness, structural issues, untranslated/uncertain entries, and package artifacts.	Generate Package; Open Package.
Project Glossary	Creates, edits, deletes, and saves exact source-to-target terminology for this application/locale.	New Term; Add Term; Delete Term; Save Project Glossary.

Tab	Purpose	Primary actions
Suggestions	Reviews terminology suggestions and confidence.	Use Suggestion; Approve High Confidence; Reject Suggestion.
Layout Proposals	Reviews direction, text fitting, columns, and geometry proposals.	Save Decision; Accept Safe All; Reset Pending. Absolute geometry remains an individual visual decision.

Review is not optional quality theater. Provider output may be grammatically valid but wrong for a short UI word. Use source context, project glossary, placeholders, and visual layout before approving terminology.

18. Wizard Step 8 - Process and Finish

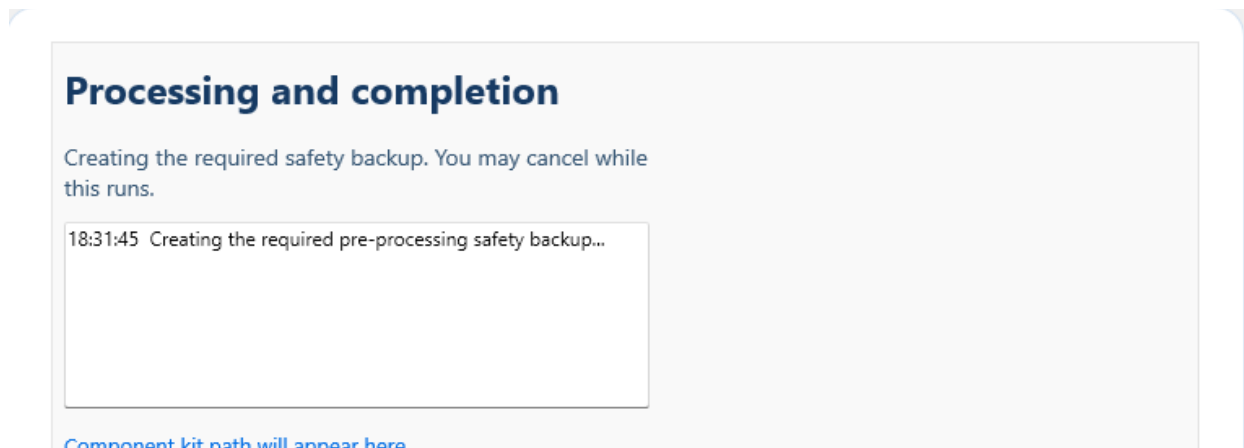


Figure 18-1. Processing status area. The final build has a simplified Finish page; obsolete command and kit-path controls are not part of the current workflow.

The progress memo records the safety backup, catalog work, translation, review return, validation, runtime-pack export, component-kit generation, detected-output deployment, optional destination deployment, and completion report. A failure is prefixed STOPPED and leaves Finish unavailable until the developer has a result to close or retry safely.

On success the page states that target Pascal, form, DPR, and DPROJ files were not edited. Back and Cancel disappear; choose Finish. The optional deployment card can deploy the application to Step 3 destinations. Leave Rebuild before deploying clear to deploy what final processing just built; select it only when another compile is actually required.

Control or area	What it does	When to use it
Progress memo	Timestamped operation and diagnostic sequence.	Read the last lines first when processing stops.
Deploy to App Folder	Copies packs to a manually chosen application folder.	Use for a one-off destination not saved in Step 3.
Rebuild before deploying	Requests another selected build before destination deployment.	Normally leave clear because final processing already compiled available target outputs.
Platform / Configuration	Chooses Win32/Win64 and Release/Debug for an optional rebuild.	Choose only combinations supported by the target project.
Deploy to Application Folders	Deploys the built application and packs to configured destinations, subject to executable authorization.	Nothing else copies the executable to separate Step 3 destinations.
Finish	Closes after successful completion.	Use even if no optional blue-card deployment was requested.
Cancel after STOPPED	Closes an unsuccessful Wizard after the diagnostic is read.	It is hidden/disabled after success because Finish is the correct action.

19. Maintenance Studio Overview

Maintenance Studio is the direct seven-page workflow: Project, Scan, Translate, Validation, Export, Integration, and Provider Settings. The left rail changes pages without rerunning completed work. Status text at the bottom reports the last action and often supplies the next required step.

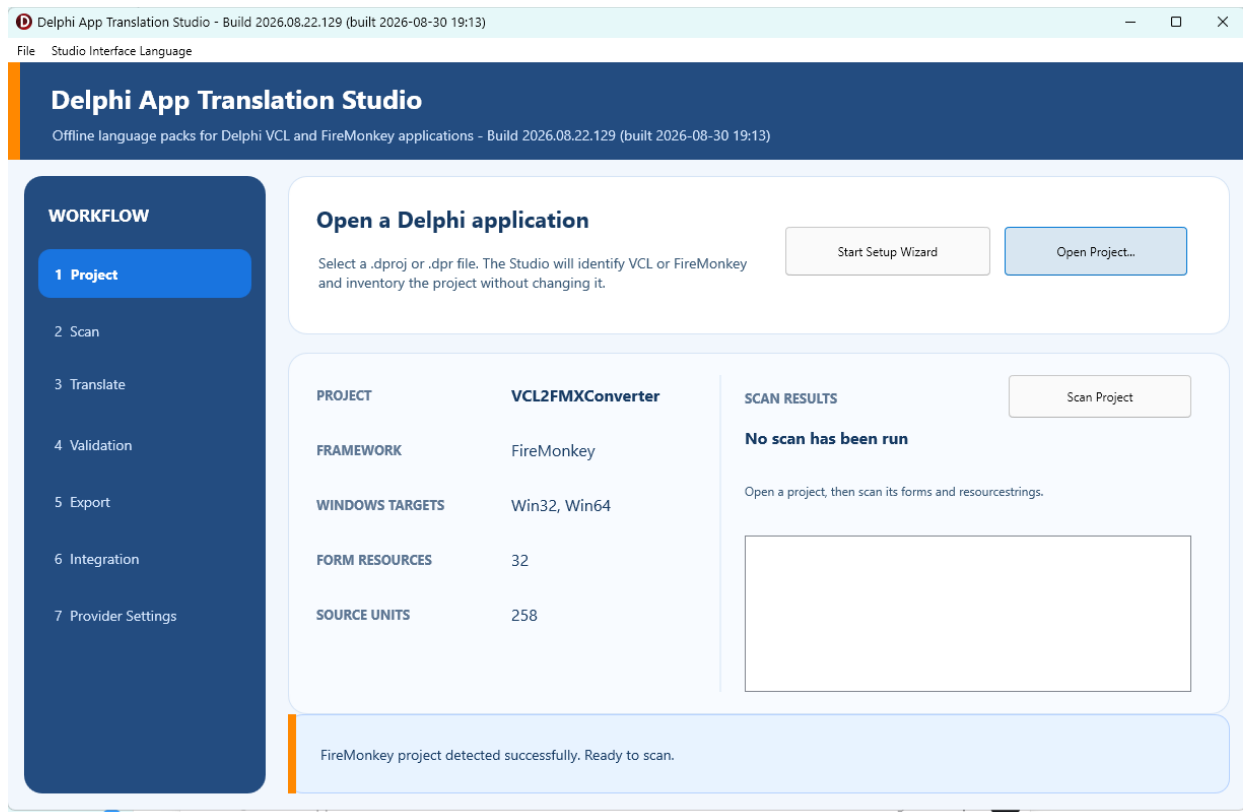


Figure 19-1. Maintenance Studio with a Delphi project loaded.

20. Maintenance Page 1 - Project

Control or area	What it does	When to use it
Start Setup Wizard	Opens the guided workflow while retaining the current project context where applicable.	Use when maintenance reveals that a complete controlled pass is preferable.
Open Project	Selects a .dproj or .dpr and inventories project metadata without editing it.	Use before Scan on a new Maintenance session.
Project details	Shows project name, framework, Windows targets, form-resource count, and source-unit count.	Verify all values before comparing scan counts.
Scan Project	Runs the same canonical scanner used by the Wizard.	Use after opening or saving changes.

Control or area	What it does	When to use it
Scan results	Shows unique entries, elapsed time, category breakdown, and observations.	Maintenance and Wizard totals should agree when project, source state, and merge basis agree.

21. Maintenance Page 2 - Scan

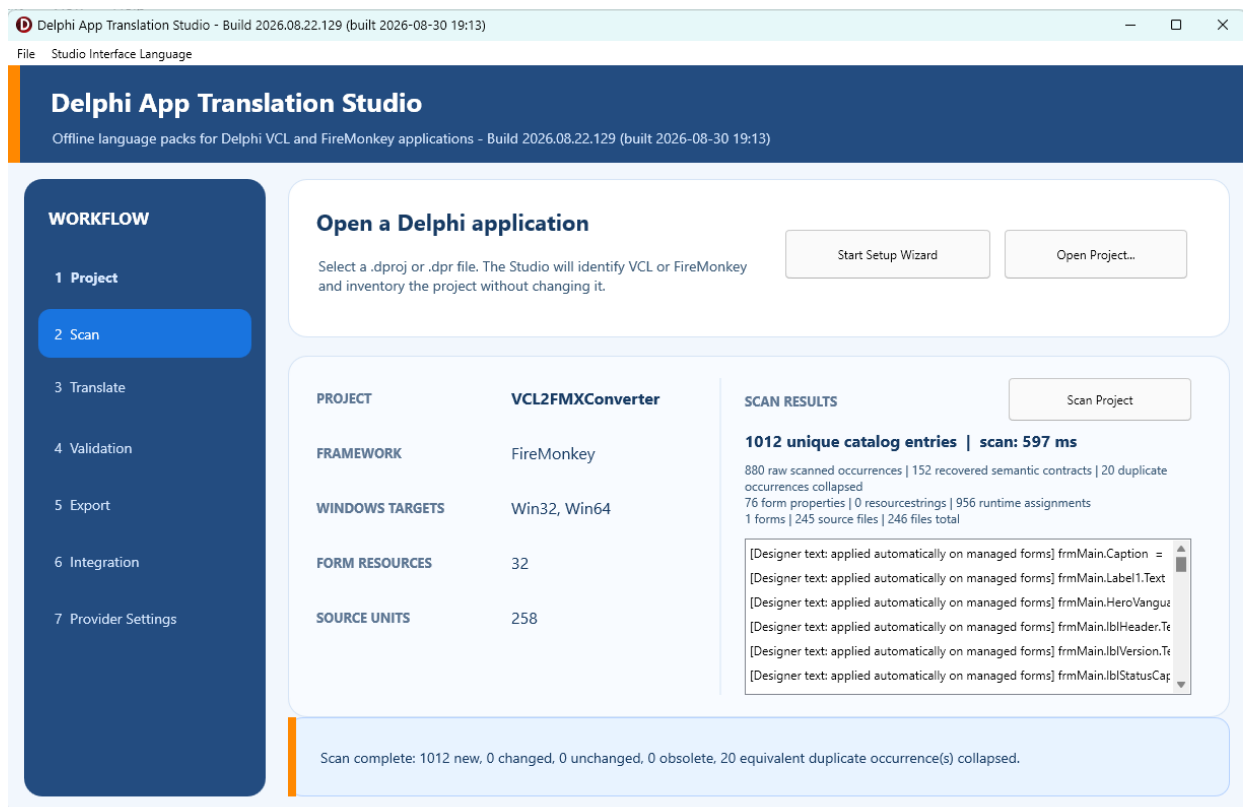


Figure 21-1. Maintenance scan showing the corrected unique/raw count contract.

The canonical scanner is shared by Maintenance and Wizard. A difference is valid only when the opened project, saved source, scanner rules, catalog merge state, or displayed category differs. The current display uses unique catalog entries as the headline and separately reports raw scanned occurrences, recovered semantic contracts, and duplicate occurrences.

A scan is read-only with respect to the selected Delphi project. Saving or merging the resulting catalog writes the Studio workspace, not target Pascal or forms.

22. Maintenance Page 3 - Translate

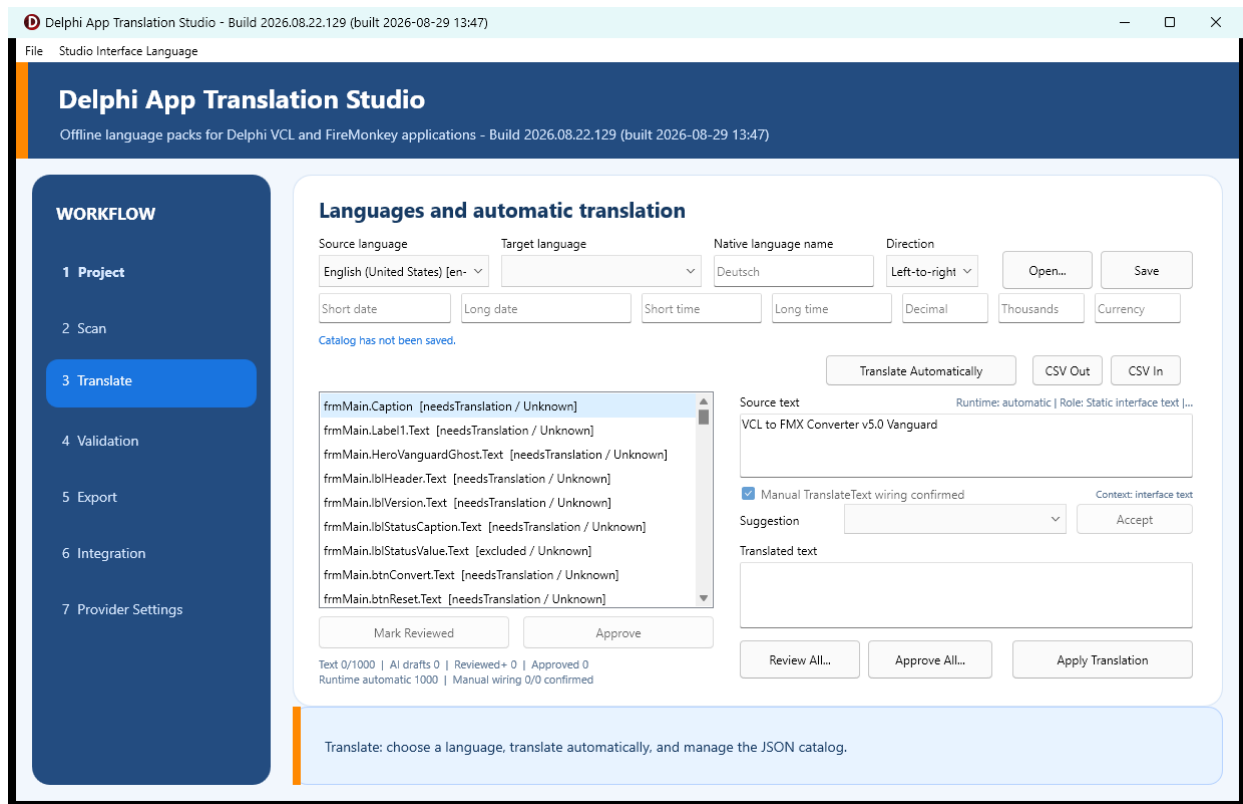


Figure 22-1. Translate page and catalog editor.

Control or area	What it does	When to use it
Source language / Target language	Select catalog locales.	Target selection is required before a new catalog is created.
Native name and locale formats	Store display name, date/time, numeric separators, and currency facts.	Review them for regional correctness.
Open / Save	Loads or atomically saves a development catalog.	Open from the workspace or an explicit compatible catalog; save after manual changes.
CSV Out / CSV In	Exports/imports translator-facing interchange.	Preserve stable keys; validate after import.

Control or area	What it does	When to use it
Entry list	Shows key plus translation/review status.	Select an entry for context and editing.
Source text	Read-only source wording and context.	Use it to judge ambiguous machine output.
Suggestion / Accept	Offers translation-memory or terminology suggestions.	Accept only after checking the exact UI meaning.
Translated text / Apply Translation	Edits and applies the selected translation.	Preserve placeholders, accelerators, line breaks, and protected tokens.
Mark Reviewed / Approve	Records linguistic review and approval for the selected entry.	Approval is a human decision, not inferred from structural validation.
Review All / Approve All	Bulk state actions.	Use only after a deliberate batch review; do not convert machine output into approval blindly.
Translate Automatically	Translates unresolved eligible entries through the configured provider.	Existing unchanged translations are preserved and stable-key/source-text recovery avoids unnecessary retranslating.

23. Maintenance Page 4 - Validation

The Validation page contains Validate the language catalog, an explanation, Run Validation, a summary, and an issue list. Double-clicking an entry-specific issue returns to the corresponding Translate entry.

Control or area	What it does	When to use it
Run Validation	Checks required translations, placeholders, accelerator keys, source changes, duplicate keys, locale/catalog metadata, and runtime structural safety.	Run after every provider pass, CSV import, glossary correction, or manual edit.
Summary	Separates errors, warnings, and informational findings.	Errors block runtime export; warnings require judgment.
Issue list	Lists key-specific and catalog-wide findings.	Double-click a key-specific item to edit it on Translate.

Structural versus linguistic status. Validation can prove that a pack is structurally safe; it cannot prove that a translation is idiomatic or correct for the product. Reviewed and Approved remain separate states.

24. Maintenance Page 5 - Export

The Export page contains Build the offline runtime pack, the current validation/readiness summary, a clickable output path, and Export Runtime Pack. The button is disabled until the catalog passes required structural validation.

Control or area	What it does	When to use it
Export summary	Reports structural readiness and separate linguistic-state counts.	Confirm the selected application ID and locale before export.
Output path	Shows the exact workspace runtime JSON path and can select it in File Explorer.	Use to inspect or manually copy a pack.
Export Runtime Pack	Writes a compact checksum-backed offline pack atomically.	Run after validation; rerun after accepted layout proposals or translation changes.

25. Maintenance Page 6 - Integration

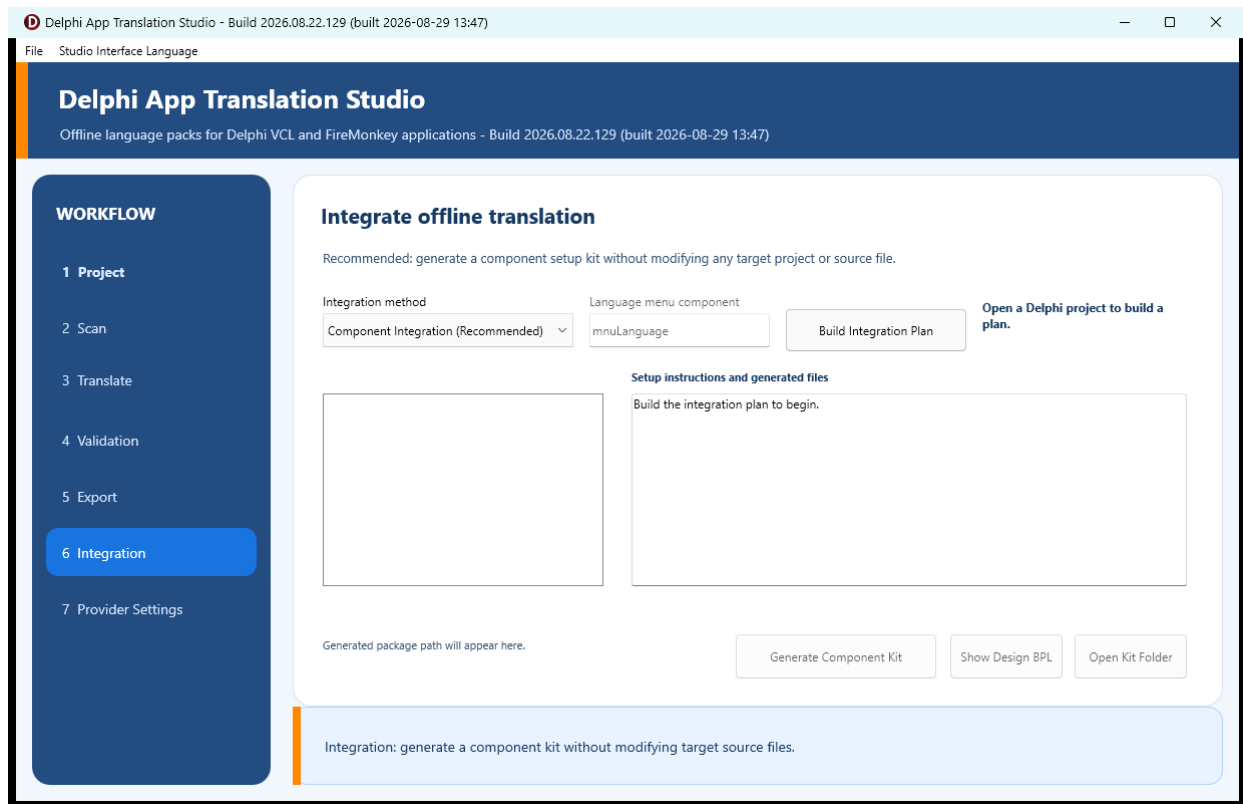


Figure 25-1. Integration page before generating a kit.

Control or area	What it does	When to use it
Integration method	Chooses Component Integration (recommended) or Automatic Source Integration (advanced).	Use Component Integration for normal projects.
Language menu component	Names an application-authored menu for advanced source integration.	Not required for the connected component selector path.
Build Integration Plan	Calculates framework, available packs, generated units/files, and intended actions without changing the target.	Always inspect the plan first.
Plan list / exact text	Shows each generated or proposed file and its content/diff.	Select each important item before generating or applying.

Control or area	What it does	When to use it
Generate Component Kit	Publishes the non-mutating component kit under export.	Recommended action.
Show Design BPL	Selects the matching compiled Win32 design BPL in File Explorer.	Use before RAD Studio Component > Install Packages > Add.
Open Kit Folder	Opens the generated kit.	Use to copy ComponentSource or inspect README/manifest/completion report.
Advanced Preview / Authorize / Apply / Restore / Complete Reset	Supports explicitly authorized source integration with preview and recovery.	Advanced only; protect the target with Git and a backup first.

26. Maintenance Page 7 - Provider Settings

Translation Setup Wizard
A safe, step-by-step path from Delphi project to offline language pack - Build 2026.08.22.129 (built 2026-08-30 19:02)

SETUP STEPS

- Welcome
- Delphi Project
- Deployment
- Languages
- Translation Service**
- Scan Project
- Review / Authorize
- Process / Finish

Connect the translation service

The Studio uses Google Cloud Translation or DeepL while the developer is online. The completed target application remains fully offline.

Provider: DeepL DeepL plan: API Free

API key:

☒ Remember securely on this computer

Save / Replace Key Test Connection A saved key is available in Windows Credential Manager

Back Next Cancel

Figure 26-1. Provider controls; the Maintenance page additionally exposes timeout, batch size, and Remove Key.

Control or area	What it does	When to use it
Provider	DeepL or Google Cloud Translation.	DeepL is default and recommended.
DeepL API plan	Free or Pro endpoint.	Choose the plan matching the DeepL key.
Timeout (seconds)	Maximum bounded provider request time; values below 5 are corrected to 5.	Increase only for a demonstrably slow connection; the default is 30.
Strings per request	Translation batch size; minimum 1.	Default 40 balances request overhead and response size.
API key	Masked new/replacement key.	A stored key is never displayed back into the field.

Control or area	What it does	When to use it
Remember securely	Credential Manager versus session-only behavior.	Use a dedicated provider key on a trusted developer computer.
Replace / Save Key	Persists settings and handles the selected provider credential.	Use after any provider, plan, key, timeout, or batch change.
Test Connection	Tests the selected provider through the bounded asynchronous operation contract.	A failure reports authentication, quota, endpoint, network, or timeout context.
Remove Key	Deletes only the selected provider's Generic Credential.	Use before transferring the computer or rotating a compromised key.

27. DeepL and Google Setup

27.1 DeepL

1. Create or sign in to a DeepL developer account at <https://www.deepl.com/pro-api>.
2. Choose API Free or API Pro. The normal consumer translator subscription is not automatically an API plan.
3. Copy the authentication key from the account's API Keys area.
4. In the Studio select DeepL and the matching plan, paste the key, choose the Remember policy, and Save / Replace Key.
5. Choose Test Connection. If the endpoint plan is wrong, switch Free/Pro and test again.
6. Review character usage and billing/quota in the DeepL account. A new key does not erase provider usage limits tied to the account/plan.

27.2 Google Cloud Translation

1. Create or select a Google Cloud project.
2. Enable Cloud Translation API Basic v2 and configure billing/quota as required by Google.
3. Create a dedicated API key.
4. Restrict the key to the Cloud Translation API and apply appropriate application/network restrictions that still permit this developer computer.
5. In the Studio select Google Cloud Translation, paste the key, save it according to the Remember choice, and Test Connection.

6. Monitor quota and costs in Google Cloud. A 401/403 usually indicates key, API enablement, billing, or restriction problems; 429 indicates quota/rate limits.

28. Runtime Integration in VCL and FMX

Responsibility	VCL	FireMonkey
Manager	TDATVCLLanguageManager	TDATFMXLanguageManager
Visible selector	TDATVCLLanguageComboBox	TDATFMXLanguageComboBox
Applicator	DAT.Runtime.VCL	DAT.Runtime.FMX
Later forms	Additive application notifications plus explicit ApplyToForm before Show/ShowModal when first-paint certainty is required.	Additive before-show lifecycle and open-form reapplication.
Direction	Bidirectional/layout state restored from designer baseline and reapplied per language.	Designer geometry snapshot restored before each LTR/RTL transformation.

The manager discovers validated packs in Localization\Languages, checks application ID/framework/version/locale/checksum/source-pack compatibility, loads the saved preference, and falls back to the validated source language when a stale preference cannot be admitted.

Selecting a language retranslates open forms immediately and persists the locale. English is a generated source pack, so switching back restores scanned English text without restarting. Writable control state, focus, selections, list selection, and live data are preserved by the runtime contract.

29. Build, Deploy, and Verify

1. Confirm the matching design package is installed and the target primary form contains one manager plus a connected selector.
2. Confirm ApplicationId, LanguagesFolder, SourceLanguage, and any FormIdentityMappings in Object Inspector.
3. Confirm the complete current dependency set is in the selected ComponentSource or project-local dependencies folder.
4. Confirm the Delphi Compiler Search Path resolves that exact folder for every supported configuration/platform.

5. Copy or deploy the canonical English source pack and every translated pack to <Executable Folder>\Localization\Languages.
6. Clean and build Win32 Debug and Release. Build Win64 only when the application will ship it and the dependency/toolchain path is valid.
7. Run from the actual output folder, not an older copy elsewhere.
8. Test startup in English, every translated locale, switching between two non-English locales, switching RTL to LTR and back, switching back to English, closing/restarting, and a stale/missing preference.
9. Open every form, dialog, menu, tab, HTML/report page, grid, long paragraph, status area, dynamic message, and error path. Check alignment, clipping, wrapping, readable font size, accelerator keys, placeholders, mixed-language remnants, and input state.
10. Repeat the matrix for each shipped platform/configuration and record defects by locale, screen, source key, expected text, actual text, and screenshot.

30. RTL, Layout, HTML, and Dynamic Text

- Direction is applied from an immutable designer baseline on every switch. It must not accumulate from the previous language.
- RTL languages right-align appropriate paragraphs and reverse visual flow where Windows/framework contracts require it; technical tokens such as WinAPI, identifiers, paths, and numbers remain protected/bidirectional as appropriate.
- Automatic fitting measures text, preserves a control that already fits, grows only into verified free space, lowers font size only to a readable floor, and wraps only as a final fallback.
- Relative, reversible layout proposals may be accepted safely in bulk. Absolute positions, absolute sizes, and grid widths require individual visual review.
- Localized HTML translates visible prose while preserving markup, comments, scripts, styles, protected nodes, product identifiers, and technical tokens. Direction and safe wrapping/padding are applied by the document contract.
- External JSON is not scanned merely because it contains strings. Project-owned operator prose is scanned only through a validated dat-translatable-resources.json manifest naming project-contained directories, file patterns, and exact string properties.
- Dynamic UI text should use stable semantic translation keys. Do not key a changing sentence only by its current English rendering when the application can identify its semantic role.

31. Security, Privacy, and Recovery

- Send only confirmed user-interface source strings to the chosen provider. Do not place secrets, customer data, personal data, or regulated data in translatable UI source.
- Use a dedicated restricted provider key; rotate it immediately if exposed.

- provider-settings.json contains no API key. Remembered keys are Generic Credentials; session keys exist only in process memory.
- Catalog, pack, glossary, settings, preference, and integration publication uses staging/validation/atomic replacement and recoverable previous files.
- Component kits are staged, validated, and published under the Studio export folder. The recommended path never copies provider code or keys into the target.
- Diagnostics should identify operation, artifact, recovery/fallback, and corrective context without logging provider credentials or raw authenticated responses.

32. Troubleshooting

Symptom	Cause to check	Corrective action
Project will not compile: DAT unit not found	Search Path points to no ComponentSource/dependency folder or an incomplete set.	Populate the complete framework set and add the exact folder under All configurations/All platforms.
Design package will not install	Wrong RAD Studio version, wrong platform, stale BPL, or .dpk selected through the wrong dialog.	Rebuild with Studio 37.0; use Component > Install Packages > Add and the Win32 Release design BPL.
Language selector is empty	No valid packs, wrong folder, wrong applicationId/framework/checksum, missing source pack, or stale executable copy.	Deploy canonical English plus translated packs under the running EXE's Localization\Languages and rebuild/run the correct output.
Switching back to English leaves translated text	Missing/incompatible source pack or runtime text lacks a stable semantic key.	Regenerate the canonical English pack and catalog the dynamic contract.
Mixed languages after switching	Previous-language runtime text was not rebuilt or a form/content surface is outside manager refresh.	Add/repair semantic runtime keys and the form/content language-change refresh contract; do not add a replay timer.
Provider test never completes	Network/API call lacks a valid endpoint or timeout path.	Confirm plan/key/network and the bounded timeout; read the returned status instead of repeatedly clicking.

Symptom	Cause to check	Corrective action
401 or 403	Invalid key, API disabled, billing restriction, endpoint-plan mismatch, or key restriction.	Correct provider setup and test again.
429	Quota or rate limit.	Wait, reduce batch size, or increase provider quota/plan.
Wizard and Maintenance counts differ	Different project/source state, old headline semantics, catalog merge basis, or scanner version.	Confirm identical project and current build; compare unique entries, raw occurrences, recovered contracts, and duplicates separately.
Export disabled	Validation has not passed.	Run Validation and correct all errors.
RTL text is left-aligned or controls overlap	Container/paragraph direction or baseline geometry contract is missing for that surface.	Record the exact screen/control; correct the universal contract, then test RTL->LTR->RTL rather than adding a locale-specific coordinate fix.
A current JSON file is rejected	Truncated/corrupt write or incompatible schema.	Preserve the .corrupt file, inspect .previous recovery, and regenerate from a valid source/catalog.
A remembered key appears missing	Provider selection changed, credential removed, or session-only mode used.	Select the correct provider and inspect Windows Credential Manager target DelphiAppTranslationStudio/Provider s/<Provider>.

33. Complete First-Time Checklist

- [] Entire repository downloaded/extracted or cloned; folder structure intact.
- [] Studio builds and starts from the intended bin output.
- [] Pristine target backup exists; disposable test copy builds before localization.
- [] Correct VCL/FMX design BPL installed through Install Packages.

- [] One manager and one connected selector saved on the primary form.
- [] ApplicationId, LanguagesFolder, and SourceLanguage verified.
- [] Target project closed before final processing.
- [] DeepL/Google key saved with the intended security policy and connection tested.
- [] Source and target locales verified; scan counts reviewed.
- [] Required safety ZIP, project-closed confirmation, and authorization reviewed.
- [] Localization Review completed; glossary and layout decisions saved.
- [] Validation has no blocking errors; runtime packs exported.
- [] Component kit generated; complete ComponentSource vendored if using dependencies.
- [] Delphi Search Path contains the exact dependency source folder for all shipped targets.
- [] Canonical English and every translated JSON pack deployed beside each executable.
- [] Full LTR, RTL, switch-back, restart, dynamic-text, HTML/report, and platform matrix passed.
- [] Intended files committed to Git; API keys and proprietary catalogs excluded.

34. Quick Path Reference

Purpose	Path
Studio repository	C:\New Delphi Projects\Delphi App Translation (example)
RAD Studio environment	C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat
Win32 compiler	C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\dcc32.exe
Studio user settings	%LOCALAPPDATA%\DelphiAppTranslationStudio
Per-project workspace	%LOCALAPPDATA% \DelphiAppTranslationStudio\Workspaces\<ApplicationId>
Target preference	%LOCALAPPDATA%\<ApplicationId>\language.ini
Generated component kit	<Studio>\export\component-integration\<ApplicationId>
Optional vendored dependencies	<Target Project>\dependencies\DelphiAppTranslation\source
Deployed packs	<Target EXE Folder>\Localization\Languages
Guides	<Studio>\docs\guides and <Studio>\docs\pdf